



Department of Computer Science

Intelligent Fake News Detection & AI Reasoning System

Using Natural Language Processing, Machine Learning, and Agentic AI

Submitted in partial fulfilment of the requirements for
Bachelor of Technology in Computer Science & Artificial Intelligence
(Course: Generative AI)

Submitted By

Name	Enrollment No.
Vikash Kumar	2401010503
Piyush Raj	2401010328
Mohammed Yaseen	2401010281
Sankalp M Tellur	2401010416

Submitted To: Kartik Gupta | **Department:** Computer Science

Academic Year: 2025–2026 | **Submitted:** 2 March 2026

 **Live Demo:** fake-news-detection-ml-ai.streamlit.app

 **GitHub (Vikash):** github.com/devVIKASHk/fake-news-detection

 **GitHub (Piyush):** github.com/Piyush-08-bot/fake-news-detection

Table of Contents

#	Section
1	Problem Statement
2	Data Description
3	Exploratory Data Analysis
4	Methodology (Classical ML)
5	Model Training Workflow
6	Evaluation
7	Optimisation
8	Advanced Agentic AI Layer
9	Application Architecture
10	Conclusion
11	Team Contribution

1. Problem Statement

Fake news is not a new problem, but the internet has made it far harder to control. A single misleading article can reach millions of people within hours through social media, WhatsApp forwards, and news aggregators. By the time a correction is published, the damage is already done.

While earlier efforts in NLP have successfully built classification algorithms that detect fake patterns, these models suffer from a "black box" problem: **they can tell you if a news article is fake, but they cannot explain *why*, nor can they verify facts.**

The core challenge we tackled was building a two-phase hybrid system:

1. **The Triage Engine:** Can a machine learning system read a news article and determine whether it is Real or Fake — purely based on the words used in it?
2. **The Investigator:** Can an advanced AI Agent parse flagged articles, break them into conceptual claims, verify those claims against the live web, and provide human-readable explanations?

Our goals were to:

-  Train a highly accurate classical ML model to detect fake news language patterns.
-  Build a LangGraph state machine to orchestrate LLM reasoning.

-  Connect the agent to the live internet for active fact-checking.
-  Deliver an interactive web dashboard providing extreme transparency to the user.

2. Data Description

2.1 Dataset Source

We used the **ISOT Fake News Dataset**, created by the Information Security and Object Technology (ISOT) Lab at the University of Victoria.

 **Dataset Link:** [ISOT Fake News Dataset on Kaggle](#)

Source	Description
Real News	Scraped from Reuters.com — a globally trusted wire news service
Fake News	Collected from sites flagged by PolitiFact and other fact-checkers
Time Period	2016–2017 (heavily focused on the US Presidential Election)

2.2 Dataset Size

Class	Count	Share
 Real Articles	21,417	55.4%
 Fake Articles	17,229	44.6%
Total	38,646	100%

Why this balance matters: A near-balanced dataset means the model cannot just guess "Real" every time and score well. It is forced to actually learn distinguishing patterns.

2.3 Features Used

The dataset had columns for `title`, `text`, `subject`, and `date`. We only trained on:

Column	Used?	Reason
<code>text</code>	✓ Yes	The full article body — our primary input
<code>label</code>	✓ Yes	0 = Fake, 1 = Real
<code>subject</code>	✗ No	Data leakage — it perfectly separates classes without learning real language
<code>title</code> / <code>date</code>	✗ No	Metadata, not article language

3. Exploratory Data Analysis

Before building any model, we explored the data to understand its structure and patterns.

3.1 Class Balance

The dataset is **55% Real / 45% Fake** — close enough to balanced that no special resampling (like SMOTE) was needed.



3.2 Article Lengths

We measured article lengths in characters and found a notable difference:

- 📰 **Real articles** — consistent length (~2,500–3,500 chars). Wire journalism follows editorial standards.
- 💣 **Fake articles** — highly variable. Ranges from ultra-short clickbait posts to very long conspiracy pieces.

Insight: Article length alone is not enough to classify news, but it confirms that fake and real content have different structural habits.

3.3 Top Words in the Dataset

After cleaning the text, the most frequent words across both classes:

Rank	Word	Notes
1	said	Journalism attributes quotes to people
2	trump	Election-era dataset
3	president	Heavy political focus
4	us	US-centric news
6	reuters	⚠️ Almost exclusively in Real articles — a data bias

⚠️ **Important Note:** The word "reuters" being this frequent is a **dataset bias**. Real articles in this dataset are mostly Reuters wire stories. The model will learn to associate this word with Real news, which may not generalise well to other real news sources.

3.4 Word Signals by Class

When we examined the model's learned weights, we found two distinct vocabulary clusters:

Signal	Words
● Real News words	reuters, washington, official, statement, government
● Fake News words	video, image, share, watch, emotional & sensational terms

This makes intuitive sense — credible journalism uses attribution language, while fake articles use engagement-bait vocabulary.

4. Methodology (Classical ML)

4.1 Text Preprocessing Pipeline

Raw news text cannot go directly into a model. Every article passed through this cleaning pipeline, defined in `utils.py`:

Raw Text

- |
- └ Step 1 → Lowercase everything ("Trump" = "trump")
- └ Step 2 → Remove punctuation & numbers (keep only letters)
- └ Step 3 → Tokenise (split into words)
- └ Step 4 → Remove stopwords ("the", "is", "a" → gone)
- └ Step 5 → Lemmatise ("running" → "run")
- └ Step 6 → Rejoin into cleaned string

Why Lemmatisation over Stemming? Stemming can produce non-words (e.g., "runn"). Lemmatisation always returns a valid base English word, giving TF-IDF a cleaner vocabulary to work with.

4.2 Feature Extraction — TF-IDF

After preprocessing, we convert each article into numbers using **TF-IDF (Term Frequency–Inverse Document Frequency)**.

The core idea:

- A word appearing a lot in *one article* but rarely across *all articles* → **High TF-IDF score** (important word)
- A word appearing in *every article* → **Low TF-IDF score** (generic, unhelpful)

Configuration used:

Parameter	Value	Why
<code>max_features</code>	5,000	Caps vocabulary to reduce noise
<code>sublinear_tf</code>	True	Log-scale dampens very common terms
<code>ngram_range</code>	(1,1)	Unigrams only — computationally efficient

4.3 Classification — Logistic Regression vs Decision Tree

We trained and compared two models:

Factor	✓ Logistic Regression	Decision Tree
Test Accuracy	98.50%	99.71%
Overfitting Risk	Low	Higher
Confidence Scores	Smooth & reliable	Jumpy
Interpretability	High (word coefficients)	Moderate
Chosen for deployment?	✓ Yes	✗ No

🏆 **Why we chose Logistic Regression:** The Decision Tree's slightly higher accuracy (99.71%) came from memorising specific article phrases — a classic overfitting sign. Logistic Regression learns *general* linguistic patterns that hold up on new, unseen articles.

5. Model Training Workflow

Training was carried out in `News_Credibility_Training.ipynb`. The full pipeline:

① Load Data

Load `True.csv` and `Fake.csv`, assign labels (`Real=1`, `Fake=0`), and merge into one DataFrame.

② Clean Data

Remove duplicates, drop null rows, and drop `title`, `subject`, `date` columns to prevent data leakage.

③ Preprocess Text

Run every article through `preprocess_text()` from `utils.py`. Store cleaned text in a new column.

④ Split Data

80% training / 20% testing with **stratified sampling** — both splits maintain the same class ratio.

⑤ Train Models

Fit `TF-IDF + Logistic Regression` and `TF-IDF + Decision Tree` pipelines on the training set.

⑥ Evaluate

Predict on the test set. Compute Accuracy, Precision, Recall, F1, ROC-AUC, and Confusion Matrix.

⑦ Save Models

Serialise trained pipelines with `joblib` to the `models/` directory for fast dashboard loading.

```
# Production pipeline
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

model = Pipeline([
    ('tfidf', TfidfVectorizer(max_features=5000, sublinear_tf=True)),
    ('classifier', LogisticRegression(max_iter=1000))
])
model.fit(X_train, y_train)
```

6. Evaluation

All results are computed on the **held-out 20% test set** — 7,730 articles the model had never seen during training.

6.1 Performance Metrics

Metric	Score	What it means
 Accuracy	98.50%	Overall correct predictions
 Precision	97.95%	Of articles predicted Real, how many actually were
 Recall	99.34%	Of all Fake articles, how many were correctly caught
 F1 Score	98.64%	Balanced measure of Precision and Recall
 ROC-AUC	0.9987	Near-perfect discrimination ability

6.2 Confusion Matrix

	Predicted Fake	Predicted Real
Actual Fake	✅ 3,411 (correct)	❌ 88 (missed)
Actual Real	❌ 28 (false alarm)	✅ 4,203 (correct)

🚨 **The most dangerous error** is a *False Negative* — a fake article that slips through as Real. We had only **28 such cases** out of 7,730 articles, giving us a **Recall of 99.34%**. That's a very strong result.

6.3 ROC-AUC

An **AUC of 0.9987** means the model has a 99.87% chance of correctly ranking a Real article above a Fake one. Perfect would be 1.0; random guessing would be 0.5.

7. Optimisation

We made several conscious decisions to improve model quality and system performance:

7.1 Model Quality

Decision	Impact
Lemmatisation over Stemming	Cleaner vocabulary → better TF-IDF features
Dropped leakage columns	Forces model to learn real language patterns, not metadata
<code>sublinear_tf=True</code>	Prevents dominant common words from drowning out rare but meaningful ones
<code>max_features=5000</code>	Balanced capacity vs. overfitting — tested higher values with no improvement
LR over Decision Tree	More stable, smoother probabilities, generalises better

7.2 System Performance

Decision	Impact
<code>@st.cache_resource</code> on model loading	Model loads once at startup, not on every user click — much faster
<code>joblib</code> serialisation	Optimised for large NumPy arrays (TF-IDF weight matrices), faster than <code>pickle</code>

8. Advanced Agentic AI Layer

While achieving 98.5% accuracy via statistical ML is excellent, it cannot cross-reference or verify facts. To address this, we implemented an advanced Agentic AI workflow using **LangGraph** and **Groq Llama-3/Mixtral models**.

If an article passes through the fast ML classifier, its text and probability score are handed to the intelligent AI LangGraph pipeline to generate "explainability". The graph executes nodes sequentially:

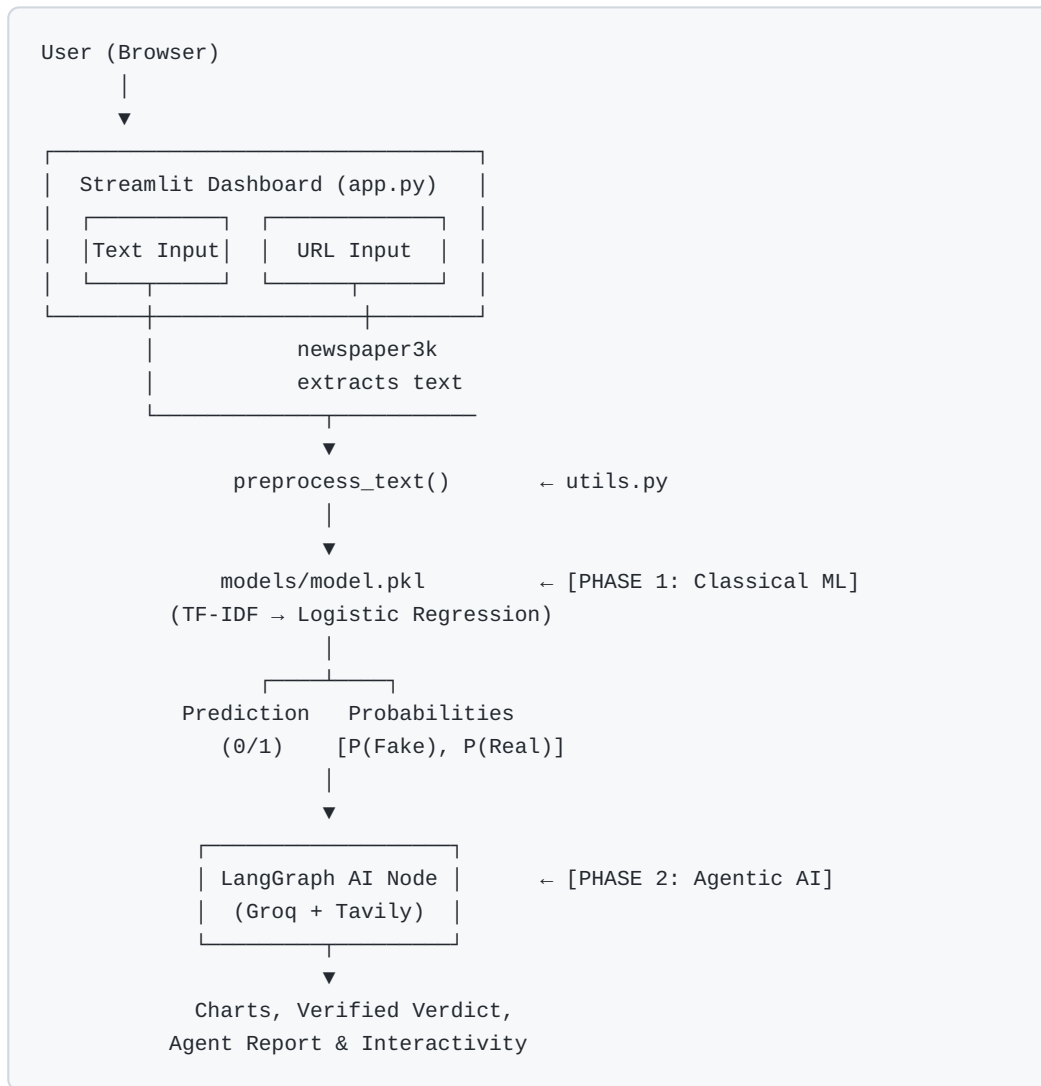
1. **Understand News (Tone & Domain):** The LLM reads the text and assigns it a Tone (Sensational vs Neutral) and a Domain (Politics, Health, etc.).
2. **Highlight Suspicious:** The LLM acts as an editor, scanning the text specifically for clickbait phrasing, extreme absolute claims, and unbacked numerical generalizations, pulling them out as exact quotes.
3. **Reason & Break Down:** The text is conceptually broken down into 3–5 isolated factual claims, assigning individual verdicts (✅ / ❌ / ?).
4. **Tool Use (Web Verification):** If the ML model flags a text as highly suspicious, the graph automatically routes the agent to use the **Tavily Web Search Tool**. The agent queries the live internet for the isolated claims and generates a verification summary citing trusted web sources.
5. **Synthesis:** The agent looks at the ML Output, Tone, Highlights, Claims, and Web Evidence, and writes a concise, bulleted explanation of *why* the article is credible or misinformation.

 **This mimics a human journalistic workflow:** Fast categorization followed by deep-dive citation checking and reporting.

9. Application Architecture

9.1 How It Works

The app is a full-stack Python web application built with Streamlit that integrates both the ML and the AI layers:



9.2 Dashboard Pages & Features

Page / Feature	What It Does
 Analysis Dashboard	Enter text or URL → live ML verdict, statistical visuals, and auto-report
 AI Sandbox	Read the AI's step-by-step reasoning (Suspicious terms, Claims, Live Web Verification).
 Comparison Tool	A standalone tool where users paste a second article for automatic side-by-side comparative credibility analysis.
 Interactive Q&A Chat	A persistent conversational interface allowing users to ask the AI agent follow-up questions about the current article.
 Model Metrics	Accuracy metrics, confusion matrix, and feature importance visualisations.

9.3 Key Files

File	Role
<code>app.py</code>	Full dashboard — UI rendering, state management, and interaction
<code>agent/graph.py</code>	LangGraph state machine orchestrating the AI thinking logic
<code>agent/nodes.py</code>	Python nodes triggering Groq and Tavily actions
<code>utils.py</code>	<code>preprocess_text()</code> — the NLP cleaning pipeline
<code>models/model.pkl</code>	Trained LR pipeline (production)
<code>News_Credibility_Training.ipynb</code>	End-to-end Machine Learning testing & training notebook
<code>.env</code>	Stored credentials for Groq and Tavily APIs

9.4 Tech Stack

Layer	Technology
ML / NLP	<code>scikit-learn</code> , <code>NLTK</code> , <code>pandas</code> , <code>numpy</code>
AI Orchestration	<code>LangGraph</code> , <code>LangChain</code> , <code>Groq API</code> (LLMs)
Web Search Integration	<code>Tavily Search API</code>
Web Framework	<code>Streamlit</code> , <code>streamlit-shadcn-ui</code>
Visualisation	<code>Plotly</code> , <code>Matplotlib</code> , <code>Seaborn</code>
URL Scraping & I/O	<code>newspaper3k</code> , <code>joblib</code>
Hosting	Streamlit Community Cloud

10. Conclusion

We started this project asking a straightforward question — can a computer tell fake news from real news just by reading it?

The answer is overwhelmingly yes. Using TF-IDF vectorisation combined with Logistic Regression, we achieved **98.50% accuracy** and an **AUC of 0.9987**.

However, we took the project significantly further by breaking the "black box" standard. By chaining the ML output directly into a **LangGraph Agentic workflow** armed with live

website tracking, we created an engine that mimics a human editor — it spots linguistic anomalies instantly, isolates suspicious text, hops onto a search engine to verify citations, and finally provides a deeply explanatory, cohesive report to the end user.

Future Improvements

- 🤖 Fine-tune **BERT or RoBERTa** for deeper semantic understanding
- 🌐 Add **multilingual support** for non-English news processing through LLM translation hooks
- 🔍 Build in a direct API with PolitiFact or Snopes alongside Tavily for hardline verification
- 🔁 Set up a **continuous retraining loop** as new misinformation patterns emerge

Overall, this project provided profound hands-on experience bridging the gap between classical structured Machine Learning protocols and modern Generative Gen-AI orchestration frameworks.

11. Team Contribution

Name	Enrollment No.	Contributions
Vikash Kumar	2401010503	Dataset collection, text preprocessing pipeline (<code>utils.py</code>), ML model training notebook, foundational AI Agent LangGraph integration & orchestration architecture.
Piyush Raj	2401010328	Backend application logic, integrating the trained model with the Streamlit dashboard (<code>app.py</code>), LangGraph node algorithms (Highlights, Claim Breakdown, Tavily Tool Use), Session State architecture handling.
Mohammed Yaseen	2401010281	Exploratory Data Analysis, model evaluation and comparison, writing and organising the academic project report.
Sankalp M Tellur	2401010416	Frontend UI design and development, GitHub repository setup, deployment to Streamlit Cloud, integration testing and QA.

All members collaborated on system design, integration testing, and final presentation of the application.

Submitted as part of the academic curriculum. All reported metrics are computed on held-out test data not used during model training.